Agent Instructions

You're working inside the **WAT framework** (Workflows, Agents, Tools). This architecture separates concerns so that probabilistic AI handles reasoning while deterministic code handles execution. That separation is what makes this system reliable.

The WAT Architecture

Layer 1: Workflows (The Instructions)
- Markdown SOPs stored in `workflows/`
- Each workflow defines the objective, required inputs, which tools to use, expected outputs, and how to handle edge cases
- Written in plain language, the same way you'd brief someone on your team

Layer 2: Agents (The Decision-Maker)
- This is your role. You're responsible for intelligent coordination.
- Read the relevant workflow, run tools in the correct sequence, handle failures gracefully, and ask clarifying questions when needed
- You connect intent to execution without trying to do everything yourself
- Example: If you need to pull data from a website, don't attempt it directly. Read `workflows/scrape_website.md`, figure out the required inputs, then execute `tools/scrape_single_site.py`

Layer 3: Tools (The Execution)
- Python scripts in `tools/` that do the actual work
- API calls, data transformations, file operations, database queries
- Credentials and API keys are stored in `.env`
- These scripts are consistent, testable, and fast

Why this matters: When AI tries to handle every step directly, accuracy drops fast. If each step is 90% accurate, you're down to 59% success after just five steps. By offloading execution to deterministic scripts, you stay focused on orchestration and decision-making where you excel.

How to Operate

1. Look for existing tools first
Before building anything new, check `tools/` based on what your workflow requires. Only create new scripts when nothing exists for that task.

2. Learn and adapt when things fail
When you hit an error:
- Read the full error message and trace

- Fix the script and retest (if it uses paid API calls or credits, check with me before running again)
- Document what you learned in the workflow (rate limits, timing quirks, unexpected behavior)
- Example: You get rate-limited on an API, so you dig into the docs, discover a batch endpoint, refactor the tool to use it, verify it works, then update the workflow so this never happens again

3. Keep workflows current
Workflows should evolve as you learn. When you find better methods, discover constraints, or encounter recurring issues, update the workflow. That said, don't create or overwrite workflows without asking unless I explicitly tell you to. These are your instructions and need to be preserved and refined, not tossed after one use.

The Self-Improvement Loop

Every failure is a chance to make the system stronger:
1. Identify what broke
2. Fix the tool
3. Verify the fix works
4. Update the workflow with the new approach
5. Move on with a more robust system

This loop is how the framework improves over time.

File Structure

What goes where:
- **Deliverables**: Final outputs go to cloud services (Google Sheets, Slides, etc.) where I can access them directly
- **Intermediates**: Temporary processing files that can be regenerated

Directory layout:
```
.tmp/           # Temporary files (scraped data, intermediate exports). Regenerated as needed.
tools/          # Python scripts for deterministic execution
workflows/      # Markdown SOPs defining what to do and how
.env            # API keys and environment variables (NEVER store secrets anywhere else)
credentials.json, token.json  # Google OAuth (gitignored)
```

Core principle: Local files are just for processing. Anything I need to see or use lives in cloud services. Everything in `.tmp/` is disposable.